Z215: Zero-Valued Spare Blocks and a Two-Truth Startup

A materially different 215-state binary machine for Con(ZFC)

Research prepared for Vladimir Reshetnikov's ProveIt repository
ChatGPT research collaboration; inherited work attributed in Section 10

21 October 2026

Abstract

We construct a concrete 215-working-state, two-symbol Turing machine whose blank-tape nonhalting is, by an ordinary proof formalizable in ZFC, equivalent to Con(ZFC). Relative to the accepted primary K215 table, the new machine changes its initial state from 0 to 74 and changes one write bit, at native row $(198, 0)$, from one to zero. This small table change alters an unbounded physical invariant: every source increment appends a zero-valued spare block of length two instead of a unit-valued block of length three. The unchanged symbolic runtime checker proves all 6,176 guarded primitive cases under the new policy.

The new blank startup reaches a source seed with stack $5 = P(0, P(0, 1))$, containing a hidden falsity code. Its calibration needs a new theorem: the actual variable-field Cantor enumeration, starting at candidate 1, has conservative stack debt at most two. A diagonal induction with an explicit finite base proves that the hidden tail is never read. This extends the class of usable startup configurations beyond stacks whose entire decoded contents are theorems.

A second fully checked 215-state construction combines a previously proposed refill scheduler with a different command order. On the formalism side, a 22-node certificate derives identity, weakening, and Peirce's law from subterm-bounded instances of the dominated Łukasiewicz row without a truth constant or zero alias. This last result is deliberately partial: it does not establish full calibration of an alternative implication encoding. No 214-state construction or state lower bound is claimed. The archive includes exact machines, sources, deterministic reconstruction, compressed runtime evidence, independent audits, proof text, and all six command-order search statuses.

Contents

1 Result, exact identity, and scope **3**

2 The zero-spare binary machine **4**
- 2.1 Exact table and state conventions . . . 4
- 2.2 Physical frames and the spare policy . . . 4
- 2.3 The complete blank prefix . . . 5

3 The inherited numerical calculus and exact source interface **6**
- 3.1 Raw codes, the dominated row, and the imported theorem . . . 6
- 3.2 The four arithmetic commands . . . 7
- 3.3 Strong compilation and cofinal completeness . . . 8

4 Protecting an arbitrary stack tail with two truth entries **8**
- 4.1 The exact schedule and its conservative stack balance . . . 9
- 4.2 An exact finite base . . . 10
- 4.3 Two prefix-sum transforms under unpairing . . . 11
- 4.4 The induction over all later diagonals . . . 12
- 4.5 Soundness above an arbitrary retained tail . . . 13
- 4.6 Completeness and the actual startup . . . 14

5 Combining the source theorem and the exact binary machine **14**

6 A second verified 215-state construction: refill scheduling and command order **15**

7 An implication fragment independent of the truth alias **16**

8 Reproduction, evidence, and proof boundaries **18**

9 What changes the search for 214 states **19**

10 Provenance and attribution **20**

A Complete dense transition table **21**

1 Result, exact identity, and scope

The principal deliverable is **Z215**. It has 215 working states, two tape symbols, a two-way infinite initially zero tape, and one designated halting state, which is not counted among the working states. Every working state has a transition for each symbol. The initialization is part of the machine: there is no prewritten input and no uncounted setup state.

The immediate parent is the accepted primary K215 at repository commit `32498004963846db7655eedce7e94f8c88f6fa8a` [1, 2]. The complete native edit is:

Item	Parent K215	Z215
Initial native state	0	74
Row $(198, 0)$	write 1, move L, enter 40	write 0, move L, enter 40
All other transition rows	identical	

The native dispatcher is 84 and the native halt label is 254. Native labels are historical, not the state count. The supplied dense presentation renames the initial state to 0, has working states $0, \ldots, 214$, dispatcher 83, and halt 215. Both presentations are bound by an explicit bijection and complete startup replay.

Theorem 1.1 (Main result)**.** *For the exact binary table* `machine/z215.tm`*, the inherited numerical-calculus calibration, the new two-truth seed theorem, the complete finite blank prefix, and the quantified primitive simulation give an ordinary proof formalizable in ZFC of*

$$\mathrm{ZFC} \vdash (\neg\,\mathrm{Halt}(\mathrm{Z215}, \mathrm{blank}) \longleftrightarrow \mathrm{Con}(\mathrm{ZFC})).$$

Equivalently, the machine halts exactly when there is a finite ZFC proof of contradiction. This equivalence does not assume the consistency of ZFC.

The source program is unchanged from the accepted primary K215. Thus the main result makes a new physical representation and initialization argument, while explicitly inheriting the parent numerical calculus and strong proof compiler. It does not claim to reprove the entire historical ZFC calibration from first principles in this report. Section 3 states that imported interface precisely.

This is a material change even though only one transition's write bit changes. The parent has 189 transition rows writing one; Z215 has 188. Consequently it is not a state renaming of the parent, even allowing global tape reflection. More substantially, its physical spare-block policy differs after every source increment. The report proves a uniform statement about this policy rather than relying on a long sample run. All 215 states are reachable in the directed transition graph from the designated start; this graph fact is not a claim that every state is visited on the particular blank computation.

Exact hashes. The native table SHA-256 is

`423c755b04ed0cd5be199178686657bf721540b858d0fb8f491239622d6d99fa.`

The dense table SHA-256 is

`d7a9d557cd08ddb0ebb1960b7fed4892db64bd38af5e0c4ae3bff9a594838825.`

The unchanged source SHA-256 is

`17af207195179b2508bf91154cd00739374f8862207ca21b9a3cb9098aa6f5ba.`

The package manifest binds every delivered file. The two machine presentations are plain text: after the initial-state directive, each row is `statereadwritedirectionnext`.

2 The zero-spare binary machine

The binary construction changes the policy for allocating unused register blocks. Its source program is the accepted 4,751-instruction K215 source; the logical operations on its nine primary registers remain the same. The new initial state reaches a different source seed, whose soundness requires the two-truth-prefix theorem proved below.

2.1 Exact table and state conventions

Use one two-way infinite tape, alphabet $\{0, 1\}$, an initially zero tape, initial head position 0, and moves L, R only. Native labels are retained from the accepted K215 table. Make exactly these changes:

$$\text{initial state: } 0 \longmapsto 74, \qquad \delta(198, 0) : (1, L, 40) \longmapsto (0, L, 40).$$

Every other transition, including both rows of state 0 and the row $\delta(198, 1) = (1, L, 113)$, is unchanged. The resulting table has 215 working states and 430 rows. Native halt label 254 has no outgoing rows and is excluded from the count. All 215 working states are reachable from 74 in the finite transition graph; this graph assertion makes no claim that all states occur on the particular blank run.

For the dense presentation, list native state 74 first and the remaining working labels in increasing order, then number this list $0, \ldots, 214$. Map native halt 254 to dense halt 215. Native dispatcher 84 becomes dense dispatcher 83. The supplied canonicalization file records both directions of this exact bijection. Thus the delivered dense machine starts in state 0 and has the usual consecutive working labels.

This construction is not a state renaming of K215. The old table has 189 rows writing symbol 1; the new table has 188. That count is invariant under state renaming. More substantively, every source increment now appends a zero-valued spare rather than a one-valued spare, changing tape growth throughout an unbounded run.

2.2 Physical frames and the spare policy

Write

$$B(v) = 0\,1^{v+1}, \qquad C(v) = 1\,0\,1^{v} \quad (v \in \mathbb{N}).$$

The primary logical register order is $(N, T, W, A, D, H, X, Y, L)$. The physical file order is $(W, L, T, D, A, Y, N, H, X)$, followed by a nonempty finite list S of natural-valued spares. No theoremhood condition is imposed on these spare values.

A frame has the dispatcher at head position h, a marker 1 at $h - 1$, a binary control address p starting at h, enough following zeros to put a header 1 at $h + 64 + g$, and then the nine primary blocks and the spare blocks. Here $g \in \mathbb{N}$, $|p| \leq 64$, and the whole remaining exterior is zero. In symbols, its finite distinguished part is

$$1\; p\; 0^{64+g-|p|}\; 1\; B(W)B(L)B(T)B(D)B(A)B(Y)B(N)B(H)B(X)B(S).$$

The head is at the beginning of p; if p is empty it scans the first zero after the marker. The notation $B(S)$ means concatenation of the spare blocks. Addresses with trailing zeros are interpreted by the unchanged binary source dispatcher. In particular the empty address denotes the source root.

Let the frame projection forget h, g, S and retain the resolved source instruction together with the nine logical registers. The change at198/0 affects only the newly allocated spare. Under the accepted engine it is $B(1) = 011$; here it is $B(0) = 01$. Each corresponding increment frame is

therefore one tape cell shorter at the end of its file. No primary value or source successor is changed by this allocation choice.

Starting from otherwise matched frames, after k source increments the newly appended spare region has $2k$ cells and k ones, instead of $3k$ cells and $2k$ ones. Thus that region is exactly one third shorter. This comparison is at corresponding source boundaries; it is not a claim about total tape space at equal binary times or about runtime speed.

Lemma 2.1 (Quantified primitive simulation)**.** *For every primitive occurrence of the pinned source, every applicable decrement guard, every natural primary register tuple, every natural gap, and every nonempty finite natural spare list, the corresponding Z215 frame executes a finite positive number of binary transitions to its prescribed successor frame. An increment appends one zero spare. Other nonhalting primitives preserve the spare list. A source halt reaches the binary halt; no nonhalting primitive block reaches it.*

Proof. A fresh execution of the supplied symbolic prover derives all 6,176 guarded cases for the 4,751 primitive occurrences. The saved per-case records are receipts of that execution, not a dump of every derivation event; the complete process is reproducible with the included code. It uses the unchanged inherited checker, with its existing allocation parameter set to zero. The argument represents complete tapes by finite symbolic words, unary runs of affine length, finite lists of blocks, and zero exteriors.

Its elementary rules are literal transition steps, induction over a unary run when the exact row is a scan-direction self-loop, and induction over a finite block list after a one-block lemma has been derived from the exact table. The four block lemmas have native entries $64, 0, 40, 97$: respectively rightward B preservation, rightward B-to-C conversion, leftward B preservation, and leftward C-to-B conversion. Each crosses one arbitrary block of value v in exactly $v + 2$ steps. Concatenation uses the elementary identity $C(S)1 = 1B(S)$ and its reversal. These are unary and list inductions, not tests on a bounded set of register values.

Thirteen affine slots represent a constant, the gap, nine independent primary parameters, the first spare value, and the total width of the remaining spare list. A positive decrement uses one plus its parameter for the selected register; its zero branch fixes that register to zero. Every resulting duration has nonnegative integer coefficients and a positive constant. The minimum constant across the complete cases is 6.

The derivations contain 1,698,790 symbolic rule applications. Together with the four block schemas they account for all 430 literal rows. The ordered case file binds each result to its source occurrence and guard; the exact table and source hashes bind the certificates to the delivered bytes. Every case closes with the specified whole-tape endpoint, including the unbounded zero exterior, proving the stated contract. □

For clarity, the exact endpoint also tracks counter carries and head motion. If $\varepsilon \in \{0, 1\}$ is the source address-increment overflow and $d = 1$ for a positive decrement, $d = 0$ otherwise, then the successor frame has head $h - \varepsilon$ and gap $g + d + \varepsilon$. Continue instructions have no increment overflow. These changes disappear under the frame projection; they are retained in the whole-tape certificates.

2.3 The complete blank prefix

Starting at native 74 on the zero tape, the machine reaches the source-root frame after exactly 117,004 transitions. Its values are

$$q = 84, \qquad h = -4, \qquad \text{header} = 80, \qquad g = 20,$$
$$(N, T, W, A, D, H, X, Y, L) = (1, 0, 5, 0, 0, 0, 0, 0, 0).$$

There are exactly 67 spare blocks. The third and eighth have value 1; every other spare has value 0. The physical primary values are $(5, 0, 0, 0, 0, 0, 1, 0, 0)$. The complete tape has 86 one-cells, with least position -5 and greatest position 240; all other cells are zero.

These assertions are an exact finite computation. An independent audit replays every one of the 117,004 transitions in both the native and dense presentations, then reconstructs the entire terminal frame from its stated primary and spare blocks. The reconstructed set of one-cells equals the replayed set. Thus the initial configuration of the source theorem is established, including the otherwise essential dirty stack value $W = 5$; a merely plausible-looking intermediate register frame is not being used.

Theorem 2.2 (Frame projection and positive-duration lift)**.** *From the displayed blank-start frame, the projection of Z215 at successive primitive boundaries is exactly the run of the pinned source from* $(1, 0, 5, 0, 0, 0, 0, 0, 0)$*. The binary machine halts if and only if that source run halts.*

Proof. Apply Lemma 2.1 inductively. The successor spare list remains finite and nonempty, so the lemma can always be applied again. It preserves the exact logical effect and source successor, while the projection forgets harmless changes of position, gap, and spare contents. Every nonhalting primitive has a finite duration at least 6. Consequently an infinite source run yields infinitely many binary transitions, and these finite consecutive blocks exhaust its deterministic binary execution. There can be no extra binary halt between source boundaries. Conversely, a finite source run ending at its halt instruction reaches the binary halt in that instruction's finite block. Prepending the exact blank prefix establishes the equivalence from the actual zero tape. □

Combining this lift with the two-truth-prefix source theorem and inherited numerical-calculus calibration gives, by ordinary reasoning formalizable in ZFC,

$$\mathrm{ZFC} \ \vdash \ \big(\mathrm{Halt}(\mathrm{Z215}, \mathrm{blank}) \leftrightarrow \neg\, \mathrm{Con}(\mathrm{ZFC})\big).$$

Equivalently its blank nonhalting is provably equivalent to Con(ZFC). The finite symbolic certificates and their induction rules supply an ordinary mathematical proof. They are not a newly compiled exact-table Lean or Rocq theorem, nor a new internal-PA endpoint. No unconditional nonhalting or state-minimality claim is made.

3 The inherited numerical calculus and exact source interface

3.1 Raw codes, the dominated row, and the imported theorem

Use the Cantor pairing function

$$P(a, b) = \frac{(a + b)(a + b + 1)}{2} + b, \qquad I(a, b) = 4P(a, b), \qquad Q_i a = 4P(i, a) + 2.$$

Truth is code 0 and falsity is code 1. In particular $I(0, 0) = 0$; decoding is not an injective map from a conventional formula grammar. All numerical arguments below range over every natural number. No well-formedness test is added to avoid unwanted raw codes.

The inherited dominated calculus $\mathcal{D}$ has truth, exact numerical modus ponens, theorem-only generalization, the seven retained logical rows $R_1, \ldots, R_7$, the gated theory bundle, and

$$L(A, B, C, U) = I\big(I(I(C, U), A), I(I(A, C), I(B, C))\big), \qquad U \le A.$$

The accepted dominated-rho8 calibration and its all-natural theorem-set comparison give the imported equivalence

$$\mathcal{D} \vdash 1 \quad \Longleftrightarrow \quad \exists p \ \mathrm{Proof}_{\mathrm{ZFC}}(p, \bot). \tag{1}$$

The exact inherited command calibration is included as `baseline/source_calibration.tex`; the accepted primary composition proof and its provenance are also retained. The complete historical definitions of the seven rows and theory bundle remain the pinned repository definitions [3, 4]. Equation (1) is an explicit dependency, not a conclusion of the finite transition audits in this submission.

The primary source has 4,751 primitive occurrences and 202 distinct subtrees. Its register order is

$$(N, T, W, A, D, H, X, Y, L).$$

Here N is the next candidate number, T the current theorem, W the encoded stack, A the selector scratch register, D an additive counter, H the cached formula, X, Y scratch registers, and L the remaining command list. At every nonhalting command return, $A = X = Y = 0$. Only candidate fetch changes N.

3.2 The four arithmetic commands

At a command boundary abbreviate (T, W, D, H) as (t, w, d, h) and use $[x_1, \ldots, x_k; w]$ for right-nested pairing over tail w. The tag and payload field lists are

Tag	Command	Fields including tag	Extra fields
0	NIP	$[0]$	0
1	DROP	$[1]$	0
2	MP(x)	$[2, x]$	1
$c+3$	AX$(c; x, y)$	$[c+3, y, x]$	2

The variable field arity matters in the new scheduling proof. A theorem for an older source with a common payload reader does not directly apply. An independent literal source audit checks the selector branches, every access to L, the four identical field-reader control graphs, and the precise candidate/continuation conventions.

The inherited command equations are

$$\mathsf{MP}(x)(t, w, d, h) = \begin{cases} (d + x, P(0, w), d + x, 0), & I(t, d + x) = h, \\ (0, w, d + x, 0), & I(t, d + x) \neq h, \end{cases}$$

$$\mathsf{DROP}(t, w, d, h) = (h, P(t, w), d, h),$$

$$\mathsf{NIP}(t, w, d, h) = \begin{cases} \mathsf{halt}, & h = 1, \\ (t, w', d, h'), & (h', w') = \mathrm{unpair}(w), \ h \neq 1. \end{cases}$$

The NIP test occurs *before* the stack pop. The only source halt is at ordinal 4736; the first stack read on its nonhalting path is at 4737. This order is independently checked against the exact source.

For AX$(c; x, y)$ put $U = d$, $A = d + x$, $B = h + y$, $C = c$. Let $G(A, B, c)$ be zero when $c = A$ or $c = B$, and the retained theory bundle otherwise. Then

$$t' = G(A, B, c), \qquad d' = 0, \qquad h' = 0,$$
$$w' = [R_1(A, B, c), \ldots, R_7(A, B, c), L(A, B, c, U), Q_U h, t; w].$$

The h in $Q_U h$ is the incoming cache. Every raw payload satisfies $U \le A$ because $A = d + x$. AX adds ten stack entries. A successful MP adds one; an unsuccessful MP adds none. DROP adds one and NIP removes one when it does not halt.

If the current top, cache, and accessible stack entries are $\mathcal{D}$-theorems, each equation preserves this property. Successful MP is precisely numerical modus ponens, and an unsuccessful MP inserts truth. AX uses only permitted rows, theorem-only generalization, and the gated bundle. This is a statement over arbitrary natural codes, including non-decodable codes. The new work is to prove that the untrusted part of the initial stack is never accessible.

3.3 Strong compilation and cofinal completeness

The inherited strong compiler assigns a finite block $\mathcal{B}_X$ to each finite $\mathcal{D}$-derivation of X. From every $(t, w, 0, 0)$, for *arbitrary* t, w, it either halts or returns $(X, w, 0, 0)$. The arbitrary-tail quantifier is essential here. It does not require that w decode entirely into theorems.

For clarity, one key reset is

$$\mathsf{Discard} = [\mathsf{MP}(1), \mathsf{AX}(1; 0, 0)]\mathsf{NIP}^{10}.$$

When the counter and cache are zero, the first MP fails because $I(t, 1) > 0$. The next AX has $U = A = c = 1$, so its gate is zero. Its generated stack entries and saved zero top can be removed without examining w, yielding $(0, w, 0, 0)$. The inherited proof gives the remaining axiom, generalization, and modus-ponens block constructions explicitly; all calculations use the command equations above.

For a refutation block, prepend $\mathsf{AX}(0; 1, Y)$ to reset the counter and cache, then execute $\mathcal{B}_1, \mathsf{DROP}, \mathsf{NIP}, \mathsf{NIP}$. Unless a previous halt has already occurred, the last two NIPs first load and then test the falsity code. If R is the fixed encoded suffix, the resulting first-candidate numbers have the cofinal form

$$N_Y = P(3, P(Y, P(1, R))) \ge Y. \tag{2}$$

This handles any finite cutoff of candidate enumeration.

Every command starting with a positive residual list first decrements it once. All subsequent field-reader tails are no greater than their inputs. Thus every nonhalting candidate drains in finitely many commands. The numerical macros terminate on all natural inputs. Consequently enumeration starting at candidate 1 eventually reaches a successful code from (2) whenever $\mathcal{D} \vdash 1$, regardless of the contents of the retained tail.

4 Protecting an arbitrary stack tail with two truth entries

The new blank initialization reaches the logical source root with

$$(N, T, W, A, D, H, X, Y, L) = (1, 0, 5, 0, 0, 0, 0, 0, 0).$$

The stack code 5 is not a stack consisting entirely of theorems: with the inherited Cantor pairing convention,

$$P(a, b) = \frac{(a + b)(a + b + 1)}{2} + b, \qquad 5 = P(0, P(0, 1)). \tag{3}$$

It contains two truth entries above a tail whose first entry is falsity. It would therefore be incorrect to invoke the usual invariant that *every* decoded stack entry is a theorem. We prove instead that the actual scheduler never reaches that tail.

The proof concerns the current source: command order NIP, DROP, MP, with fields read only by the commands that need them. The earlier protected-tail theorem for a common field reader and order MP, DROP, NIP does not supply this conclusion. Both the command order and the field arities matter below.

4.1 The exact schedule and its conservative stack balance

Let $f(n)$ be the second component of the inverse of P. Write a freshly selected candidate as $n = P(t, y)$. Its first command and remaining proof-list pointer r are

t	command	conservative stack change	r
0	NIP	-1	y
1	DROP	$+1$	y
2	MP	0	$f(y)$
$t \geq 3$	AX	$+10$	$f(f(y))$.

(4)

For MP the first unpairing of y supplies its additive field. For AX the two unpairings supply its two fields; its remaining parameter is $c = t - 3$. NIP and DROP have no additional field. If $r > 0$, continuation fetch decrements it before decoding, so the next packed command is $r - 1$. If $r = 0$, the candidate is finished.

Define a finite command word $\mathcal{W}(n)$ recursively by taking the first command in (4), followed by $\mathcal{W}(r - 1)$ when $r > 0$. The pairing inequalities give $r \leq y \leq n$, so this recursion is well founded. In particular,

$$|\mathcal{W}(n)| \leq n + 1. \tag{5}$$

Indeed, a nonempty recursive suffix has length at most r by induction. A nonhalting source run from $N = 1, L = 0$ executes $\mathcal{W}(1), \mathcal{W}(2), \ldots$ in that order. Registers containing formulas affect MP success and possible halting, but do not change this schedule before a halt.

The conservative change is exact for DROP, AX and a nonhalting NIP. A successful MP pushes a truth entry, while an unsuccessful MP leaves the stack alone; assigning MP the value zero is therefore a lower bound. AX pushes the old top, one generalized cache formula, and eight logical rows, irrespective of whether its theory-bundle gate is open. A halting NIP is treated as having change -1 only in the abstract word: the actual run has already stopped before its pop.

Let $b(n)$ be the total conservative balance of $\mathcal{W}(n)$ and let $m(n)$ be its minimum prefix balance, including the empty prefix. If the first-command change is δ and its suffix pointer is r, then

$$b(n) = \delta + \begin{cases} b(r-1) & r > 0, \\ 0 & r = 0, \end{cases}$$
$$m(n) = \min\left(0, \delta + \begin{cases} m(r-1) & r > 0, \\ 0 & r = 0. \end{cases}\right). \tag{6}$$

Set $B(-1) = 0$ and $B(n) = \sum_{i=0}^{n} b(i)$. Since every command has change at least -1,

$$m(r-1) \geq -r \qquad (r > 0). \tag{7}$$

We use the same bound with value zero for an empty suffix.

Theorem 4.1 (Exact conservative debt)**.** *Every command prefix of $\mathcal{W}(0)\mathcal{W}(1)\mathcal{W}(2)\cdots$ has conservative balance at least -3. Every command prefix after omitting $\mathcal{W}(0)$ has balance at least -2, and this latter lower bound is attained. Furthermore, $B(n) \geq 7$ for every $n \geq 6$.*

The assertion is about this conservative accounting. It does not claim that two initial truth entries are necessary for every actual execution: successful MP commands can supply extra entries.

4.2 An exact finite base

Write $\Delta_s = s(s+1)/2$. Cantor diagonal s consists, in order, of

$$P(s-y, y) = \Delta_s + y, \qquad 0 \le y \le s.$$

The base covers the complete diagonals $0, \dots, 35$, namely the 666 candidates $0, \dots, 665$ because $\Delta_{36} = 666$.

Lemma 4.2 (Finite base)**.** *For $0 \le n \le 665$,*

$$B(n-1) + m(n) \ge -3.$$

For $6 \le n \le 665$, $B(n) \ge 7$, and $B(665) = 5762$. The first six values of $B(n)$ are $-1, 0, -2, -2, -2, -2$.

Proof. These statements follow by exact integer evaluation of (6). The following complete algorithm specifies the calculation. All array references on its right sides are to smaller indices, as proved above.

```
b = []; m = []; length = []; B = []
total = 0
for n in range(666):
    t, y = unpair(n)
    if t == 0:   delta, r = -1, y
    elif t == 1: delta, r =  1, y
    elif t == 2: delta, r =  0, second(y)
    else:        delta, r = 10, second(second(y))
    assert r == 0 or r - 1 < n
    b.append(delta + (b[r-1] if r else 0))
    m.append(min(0, delta + (m[r-1] if r else 0)))
    length.append(1 + (length[r-1] if r else 0))
    assert length[n] <= n + 1
    assert total + m[n] >= -3
    total += b[n]
    B.append(total)
assert min(B[6:]) == 7
assert B[665] == 5762
assert B[:6] == [-1, 0, -2, -2, -2, -2]
```

Here the inverse is evaluated without floating point: if $d = \lfloor (\sqrt{8n+1} - 1)/2 \rfloor$ and $e = n - \Delta_d$, then $\mathrm{unpair}(n) = (d - e, e)$. The supplied independent checker uses integer square root. Its loop invariant is exactly the definition of b, m, B and (5); thus this is a finite proof of the induction base, not a bounded replacement for the infinite assertion. □

For example, the first ten total word balances are

$$-1,\ 1,\ -2,\ 0,\ 0,\ 0,\ 10,\ 0,\ 2,\ -3.$$

Candidate 5 begins with NIP at global balance $B(4) = -2$, so its first command attains -3.

4.3 Two prefix-sum transforms under unpairing

Introduce

$$g(0) = 0, \qquad g(r) = b(r-1)\ (r > 0), \qquad G(t) = \sum_{r=0}^{t} g(r) = B(t-1).$$

All prefix sums in this subsection are inclusive unless an upper condition $y < k$ is written. The proposed lower bound for G is

$$p(0), \ldots, p(6) = (0, -1, 0, -2, -2, -2, -2), \qquad p(t) = 7 \quad (t \geq 7). \tag{8}$$

On successive complete Cantor diagonals, the sequence of second coordinates is

$$(0), \quad (0,1), \quad (0,1,2), \quad \ldots.$$

Consequently, if $t = \Delta_d + e$ with $0 \leq e \leq d$, then

$$H(t) := \sum_{y=0}^{t} g(f(y)) = \sum_{j=0}^{d-1} G(j) + G(e). \tag{9}$$

Whenever the relevant inequalities $G(j) \geq p(j)$ are available, this gives

$$H(t) \geq q(t), \qquad q(\Delta_d + e) := \sum_{j=0}^{d-1} p(j) + p(e).$$

The negative part of this explicit auxiliary sequence is small. The following table records its complete finite calculation through $t = 44$, or $d = 8$:

d	$\min_{0 \leq e \leq d} q(\Delta_d + e)$	$\sum_{e=0}^{d} \min(0, q(\Delta_d + e))$
0	0	0
1	−1	−1
2	−2	−4
3	−3	−7
4	−5	−20
5	−7	−37
6	−9	−58
7	−11	−74
8	−4	−23

For $d \geq 9$,

$$\sum_{j=0}^{d-1} p(j) = 7d - 58 \geq 5, \qquad p(e) \geq -2,$$

so $q(t) \geq 3$ for $t \geq 45$. We have therefore proved, for all t,

$$q(t) \geq -11, \qquad \sum_{t \geq 0} \min(0, q(t)) = -224. \tag{10}$$

Apply the same diagonal identity a second time. For $k > 0$, write $k - 1 = \Delta_d + e$. Then

$$\begin{aligned} S(k) &:= \sum_{y=0}^{k-1} g(f(f(y))) \\ &= \sum_{j=0}^{d-1} H(j) + H(e) \\ &\geq \sum_{j=0}^{d-1} q(j) + q(e) \geq -224 - 11 = -235. \end{aligned} \tag{11}$$

For $k = 0$ the same lower bound is immediate.

Only finite prefix hypotheses are needed for this reasoning. If $G(t) \geq p(t)$ is known for $0 \leq t \leq M$, every displayed instance with $k \leq M + 1$ follows: every argument introduced by the two diagonal decompositions is at most the original argument. This finite observation prevents any circular use of the desired global estimate.

4.4 The induction over all later diagonals

We now prove Theorem 4.1. Suppose $s \geq 36$ and the conclusions have been proved before diagonal s. In particular $G(t) \geq p(t)$ is known for $0 \leq t \leq s$: its largest required earlier index is $s - 1 < \Delta_s$. Thus (11) applies throughout the current diagonal.

Its first $s - 2$ candidates, at $y = 0, \ldots, s - 3$, are AX words. After the first k complete AX words their relative balance is

$$A(k) = 10k + \sum_{y=0}^{k-1} b(f(f(y)) - 1) = 10k + S(k) \geq 10k - 235, \tag{12}$$

where the summand is zero when $f(f(y)) = 0$. Within the next AX word, its initial command adds ten and its suffix has minimum at least $-f(f(k)) \geq -k$ by (7). For $k \geq 26$, the balance after that initial command and at every later prefix of its suffix is at least

$$10k - 235 + 10 - k = 9k + 10 - 235 \geq 0.$$

The balance before the initial command is also nonnegative because $10k - 235 \geq 0$.

For the remaining finite set $0 \leq k \leq 25$, direct evaluation using (6) verifies both

$$A(k) \geq 0, \qquad A(k) + 10 + \begin{cases} m(f(f(k)) - 1) & f(f(k)) > 0, \\ 0 & f(f(k)) = 0 \end{cases} \geq 0. \tag{13}$$

These are the 26 short AX checks in the independent finite-base checker. They involve only earlier candidate values. Hence every command prefix in the entire AX portion is nonnegative relative to the beginning of its diagonal.

The last three candidates on diagonal s are, in order, MP at $y = s - 2$, DROP at $y = s - 1$, and NIP at $y = s$. Their conservative losses, including every internal prefix, are at most

$$s - 2, \qquad s - 2, \qquad s + 1,$$

respectively. For DROP, its own $+1$ offsets one unit of the possible loss of its suffix of length at most $s - 1$. For NIP, its initial -1 is added to a possible suffix loss of s. These bounds include the

complete words as well as their prefixes, so they may be added. The total loss of the final three words is at most $3s - 3$.

After the AX portion, (12) gives balance at least $10(s-2) - 235$. Every subsequent prefix is therefore at least

$$10(s-2) - 235 - (3s-3) = 7s - 252 \geq 0$$

relative to the start of the diagonal. Thus no prefix of this diagonal falls below its initial global balance. That initial balance is at least seven by the induction hypothesis. This proves both the global prefix bound and $B(n) \geq 7$ on the entire new diagonal. Lemma 4.2 starts the induction.

Finally, $\mathcal{W}(0)$ is one NIP and has balance -1. Removing it raises every later prefix balance by one. The bound becomes -2, and the first NIP in candidate 5 attains it, as observed above. This completes the proof of Theorem 4.1.

4.5 Soundness above an arbitrary retained tail

Let $\mathcal{D}$ denote the inherited calibrated numerical calculus. It contains truth code 0, is closed under numerical modus ponens and generalization, contains the eight logical rows emitted by AX, and contains its gated theory bundle whenever the gate succeeds. A failed gate returns truth. No assumption of consistency is made.

For a finite list J of numbers and an arbitrary natural tail S, define

$$\mathrm{push}([], S) = S, \qquad \mathrm{push}(x :: J, S) = P(x, \mathrm{push}(J, S)).$$

This is a representation invariant, not a claim that numerical stack codes have a unique finite length.

Theorem 4.3 (Two initial truth entries suffice)**.** *At a genuine current-source root, suppose $N = 1, L = 0$ and the selector and scratch registers are zero. Suppose T and H are $\mathcal{D}$-theorems, D is arbitrary, and*

$$W = P(0, P(0, S))$$

for an arbitrary natural number S. Every nonhalting command preserves theoremhood of the current formula and cache, and the execution never reads S as a stack. If it halts, $\mathcal{D}$ derives 1.

Proof. Track the stack as $\mathrm{push}(J, S)$, initially $J = [0, 0]$, with every member of J a theorem. DROP pushes the theorem-valued current formula and changes the top to the theorem-valued cache. AX pushes ten theorem-valued entries and returns a theorem-valued bundle or truth, with cache zero. An unsuccessful MP returns truth; a successful MP obtains its new top by numerical modus ponens and pushes an additional truth entry.

Before any possible pop, the length of J equals two plus the conservative balance so far plus the number of successful MP commands. Theorem 4.1 bounds the conservative balance after the prospective NIP pop below by -2. Therefore its value before that pop is at least -1, and J is nonempty. A nonhalting NIP consequently reads a theorem from J, never from S. The identity $\mathrm{unpair}(P(x, z)) = (x, z)$ justifies this step even at the alias $P(0, 0) = 0$: a conceptual leading zero is tracked by the invariant even when its numerical insertion leaves the stack code unchanged.

If NIP halts before its pop, its incoming cache equals 1. That cache is a theorem by the same induction, so a native refutation exists. There is no appeal to the theoremhood of the arbitrary tail. □

4.6 Completeness and the actual startup

For completeness, the current command compiler supplies a strong finite block for each numerical derivation of a code X. From every tuple $(t, w, 0, 0)$ of top, stack, counter and cache, that block either halts earlier or returns $(X, w, 0, 0)$. The retained tail w is arbitrary in this contract; it need not be a stack of theorems.

For a derivation of 1, append DROP; NIP; NIP to its strong block. If the block has not already halted, these three commands push the newly derived 1, read it into the cache, and halt. Prepend

$$\mathrm{START}_Y = \mathrm{AX}(c = 0; x = 1, y = Y).$$

This command clears the counter and cache regardless of their incoming values, so the strong block applies above whichever stack it leaves. If R is the continuation encoding of the fixed suffix, the initial candidate code for this entire successful word is exactly

$$n_Y = P(3, P(Y, P(1, R))) \geq Y. \tag{14}$$

AX fields are ordered $[c + 3, y, x]$. Nonempty continuations have an outer successor, whereas the first command of a freshly selected candidate does not; these conventions are the reason that (14) also covers the zero NIP tag.

Every earlier candidate drains after finitely many commands unless it halts, by the strict descent of the suffix index. Arithmetic command macros terminate and preserve the next-candidate counter. Thus (14) ensures that any refutation eventually causes halting, from every finite cutoff and every retained working tuple.

Corollary 4.4 (Calibration of the new seed)**.** *The current source started at* $(1, 0, 5, 0, 0, 0, 0, 0, 0)$ *halts if and only if* $\mathcal{D}$ *derives* 1*. Through the inherited numerical-calculus calibration, its nonhalting is equivalent to* Con(ZFC)*, by ordinary reasoning formalizable in ZFC.*

Proof. Equation (3) and Theorem 4.3 give soundness with $S = 1$. The arbitrary-tail strong compiler and (14) give completeness. The inherited calibration identifies numerical refutability with a finite ZFC refutation. □

This section establishes the source-level initial-condition theorem. The complete binary runtime theorem, the exact finite blank startup, and their binding to the released transition table remain separate parts of the machine proof. The finite calculations here prove the stated induction base and constants; the diagonal induction supplies the assertion for all future candidates.

5 Combining the source theorem and the exact binary machine

The blank prefix reaches the source root with

$$(N, T, W, A, D, H, X, Y, L) = (1, 0, 5, 0, 0, 0, 0, 0, 0).$$

Since $5 = P(0, P(0, 1))$, this seed has exactly the required two truth entries above tail $S = 1$. It would be unsound to assert that all successively decoded heads of this stack are theorems: the third head is the falsity code. The two-truth theorem proves that this head is never read on any nonhalting source prefix. Initial top and cache are truth, and the counter and scratch conditions are satisfied.

If Z215 halts, its exact finite startup has not halted and the positive primitive simulation places that halt at the source NIP cache test. The accessible-prefix invariant implies $\mathcal{D} \vdash 1$. By (1), ZFC has a finite proof of contradiction.

Conversely, a ZFC proof of contradiction gives a finite $\mathcal{D}$-refutation by (1). The strong compiler and cofinality (2) give a successful enumerated command candidate above the startup cutoff. All preceding nonhalting candidates drain. The source therefore halts after finitely many primitives, and each such primitive is simulated in a positive finite number of binary transitions. Thus Z215 halts.

These two implications prove Theorem 1.1. The quantified runtime argument uses finite lists, induction over unary lengths, and affine transition counts; the schedule proof uses induction on natural Cantor diagonals. The compiler is primitive recursive on finite derivations. Each ingredient can be expressed and proved in ZFC. We do not invoke the truth of Con(ZFC) while establishing the equivalence, nor infer unconditional nonhalting from it.

6 A second verified 215-state construction: refill scheduling and command order

A separate construction combines the shared-`CB1` refill scheduler with command order `DROP/NIP/MP`. The two ingredients had appeared separately: the accepted order-120 machine used the ordinary refill scheduler, while the latest shared-refill construction used order 210. The combined machine, called `partialcb120` in the artifacts, has 215 working states and 430 transition rows. Its source has 4,752 primitive occurrences, 202 distinct subtrees, depth 47, and maximum zero run six. The full rooted, edge-labelled transition graph differs from all compared accepted 215-state machines and from the prior shared-refill machine, including after global tape reflection.

Uniform command-order transport. Let π permute the names $\mathrm{MP}, \mathrm{DROP}, \mathrm{NIP}$. Assign tag j to command $\pi(j)$ and keep axiom tags $c+3$ unchanged. The three command bodies are closed subtrees at addresses `111`, `110101`, and `110100101`. The two independent source graph resolvers check all primitive guarded edges, the three exact selector cases, and the exhaustive residual case $c \geq 3$. They also check that normal command-return wrappers reach the source root and that the axiom body and every external source path are unchanged. A proof compiler transports by replacing each command tag with its inverse-permutation tag. Every command body implements the same abstract operation over arbitrary natural register values. Thus the inherited raw-code soundness invariant is preserved, including on junk codes. The strong reset-and-compile theorem transports in the same way. Its cofinal family still has form

$$P(3, P(Y, P(1, R))), \qquad P(a,b) = \frac{(a+b)(a+b+1)}{2} + b,$$

where the fixed suffix R is replaced by its translated encoding. The axiom tag 3 is unchanged, so every finite cutoff is exceeded by a successful code. These assertions calibrate the register source; blank binary initialization remains a separate obligation.

Refill simulation. In register order $(N, T, W, A, D, H, X, Y, L)$, put

$$G = (\mathrm{CD}_8, \mathrm{CB}_1), \quad F = (\mathrm{CD}_0, (\mathrm{CI}_8, (\mathrm{CI}_6, \mathrm{CC}_3))), \quad H_0 = (\mathrm{CD}_6, \mathrm{CC}_2).$$

The old prefix is $G, (F, (\mathrm{CI}_0, H_0))$; the new one is $G, (F, (\mathrm{CI}_0, (H_0, \mathrm{CB}_1)))$. For a positive incoming list, both decrement the list once and enter the decoder. From $L = 0, N = n > 0, X = 0$, the new

prefix enters the decoder with $(N, L, X) = (n+1, n-1, 0)$. From $n = 0$ it reaches $(2, 0, 0)$ after one additional refill. These facts follow by induction on the transfer and restoring loops; the literal ten-node prefix and every tail control edge are independently checked. The resulting candidates are $n-1, n, n+1, \ldots$, or $0, 1, 2, \ldots$ in the exceptional zero-counter case. Every nonhalting candidate drains, because each command strictly decreases its positive residual list. The cofinal strong compiler proves completeness, and the unchanged theorem invariant proves soundness. Equivalently, relate the new and old counters by $N_n = N_o + 1$ at command boundaries, with all other registers identical. Both empty-list cases then enter the decoder with list N_o; the extra root visit is finite. This is a stuttering simulation.

Exact binary result. The unchanged quantified runtime prover discharges all 6,177 guarded cases, using 1,698,860 symbolic rules. It covers every one of the 430 rows; the minimum affine duration constant is nine. The domain includes all natural primary registers and gaps, arbitrary nonempty finite lists of natural spares, and the whole zero exterior. Blank execution reaches native dispatcher 84 after 88,746 transitions, with head -12, gap five, 45 unit-valued spares, 110 one cells, and

$$(N, T, W, A, D, H, X, Y, L) = (5, 0, 0, 0, 0, 0, 0, 0, 4).$$

An independent audit replays the entire prefix on both native and dense tables and reconstructs the whole terminal frame. The first source command is MP(0), because the initial list decrement gives $3 = P(2, 0)$ and tag two denotes MP. It is harmless on the zero theorem registers and returns to the same tuple with $L = 0$ after exactly 65 source primitives. The counter-offset relation now matches the already calibrated order-120 root with $N = 4$. The finite prefix, positive-duration binary simulation, refill simulation, and inherited numerical-calculus theorem therefore give an ordinary proof formalizable in ZFC of

$$\neg\mathrm{Halt}(\mathrm{partialcb120}, \mathrm{blank}) \quad \longleftrightarrow \quad \mathrm{Con}(\mathrm{ZFC}).$$

This is not a new proof-assistant kernel artifact or a 214-state result.

Negative controls. All six command orders admit 215-state allocations and pass the finite decoder-prefix gate. That gate does not prove initialization. Order 012 passes all 6,176 quantified primitive cases but did not reach a calibrated root within ten million tested blank steps. Order 021 actually halts in 31,714 blank steps and is rejected. Orders 102 and 201 reached no calibrated root within 500,000 tested steps. The new order-120 allocation passes all 6,176 cases and reaches the usual safe root in 96,308 steps; order 210 is isomorphic to an accepted composed allocation. The files state these distinctions explicitly. The search supplies no 214-state table and no lower bound.

7 An implication fragment independent of the truth alias

The inherited full dominated-ρ_8 certificate uses the literal arithmetic identity $I(0, 0) = 0$. This creates a separate issue when considering an implication constructor with a nonzero residue tag. The following new finite derivation removes that dependence for identity, weakening, and Peirce's law. It does not claim the whole classical-basis or encoding-calibration theorem.

For this section let X be any set, let $I : X \times X \to X$ be any binary operation, written $a \to b$, and let Th be a predicate on X. Define

$$L(a, b, c, u) = ((c \to u) \to a) \to ((a \to c) \to (b \to c)).$$

Theorem 7.1 (Two subterm schemas derive I, K, P)**.** *Suppose* Th *is closed under modus ponens and contains every instance of*

$$\mathrm{A}_0: \ L(u,b,c,u), \qquad \mathrm{A}_1: \ L(z \to u,b,c,u).$$

Then it contains every instance of

$$p \to p, \qquad p \to (q \to p), \qquad ((p \to q) \to p) \to p.$$

No truth constant, equation on I, injectivity, order, or interpretation of members of X as formulas is assumed.

Proof. The following is a complete 22-node schematic derivation. Its notation is Polish implication: Cuv means $u \to v$, and letters are metavariables universally quantified independently in each row. CD(i, j) means instantiate major row i and minor row j so that the minor is exactly the antecedent of the major, then apply modus ponens. Every required substitution is explicitly stored in `research/alias_free/alias_free_ikp.json`; the table gives the resulting schemas.

Node	Rule	Conclusion
0	A_0	`CCCabbCCbaCca`
1	A_1	`CCCabCcbCCCcbaCda`
2	CD$(1,0)$	`CCCCCabbCabCbCabCcCbCab`
3	CD$(1,1)$	`CCCCCabbCabCbbCcCbb`
4	CD$(1,3)$	`CCCaCbbCCCcbbCcbCdCCCcbbCcb`
5	CD$(4,0)$	`CaCCCbbbCbb`
6	CD$(5,0)$	`CCCaaaCaa`
7	CD$(1,6)$	`CCCaaCaaCbCaa`
8	CD$(7,7)$	`CaCCbbCbb`
9	CD$(8,0)$	`CCaaCaa`
10	CD$(7,9)$	`CaCbb`
11	CD$(10,0)$	`Caa`
12	CD$(1,11)$	`CCCabaCca`
13	CD$(1,12)$	`CCCabCbcCdCbc`
14	CD$(2,12)$	`CaCbCcb`
15	CD$(14,0)$	`CaCba`
16	CD$(1,13)$	`CCCaCbcCdbCeCdb`
17	CD$(16,0)$	`CaCCCCbcdcCbc`
18	CD$(17,0)$	`CCCCabcbCab`
19	CD$(1,18)$	`CCCabCCabcCdCCabc`
20	CD$(19,12)$	`CaCCCbcbb`
21	CD$(20,0)$	`CCCabaa`

Rows 11, 15 and 21 are the three asserted schemas. The independent forward checker evaluates all stored substitutions and checks exact antecedent and consequent equality at each of the twenty modus-ponens nodes. It checks the two axiom shapes structurally. There is no implication normalization, truth leaf, unification, or search in that checker. The resulting finite derivation is valid under every valuation in X: equal formal expressions have equal evaluations, whether or not distinct expressions can have the same value. □

Corollary 7.2 (Application to every nonnegative implication tag)**.** *Let P be Cantor pairing and let $I_j(a,b) = 4P(a,b) + j$ with $j \geq 0$. Any numerical calculus closed under modus ponens and containing $L(A,B,C,U)$ whenever $U \leq A$, using I_j as implication, derives every numerical instance of identity, weakening, and Peirce.*

Proof. For A_0, the restriction is $u \le u$. For A_1, it is $u \le I_j(z,u)$. Cantor pairing dominates each coordinate, and hence $I_j(z,u) \ge u$. Theorem 7.1 now applies directly to $X = \mathbb{N}$. Its argument includes every natural assignment, including zero, aliases, and codes not decoded by a conventional formula grammar. □

In particular, this corollary applies to the proposed $I_3/R_1/Q_0/E_2$ constructor assignment, in which the old equation $I(0,0) = 0$ is absent. It establishes three exact logical interfaces for that encoding without translating the numerical order or the shared formula-code/variable-label namespace.

The remaining gap is explicit. This certificate does not derive distribution S, composition B, or the unrestricted four-argument L schema. The earlier full certificate therefore cannot yet be replaced by this fragment to obtain the alternate encoding's complete consistency calibration. The unresolved derivations are further research questions; incomplete searches are not evidence that they are impossible. The released zero-spare machine retains the inherited implication constructor and its existing full calibration, so its correctness does not depend on resolving this separate question.

The proof search adapted the repository's existing condensed-detachment helpers. The new contribution is the two-schema derivation, its explicit-substitution certificate, and its independent forward checker; the search infrastructure is inherited.

8 Reproduction, evidence, and proof boundaries

The archive is designed to be placed under the repository's research directory. The ordinary proofs are in this report and the retained Markdown/LaTeX sources. Exact table identity and the finite premises have separate executable checks. Python 3.10 or later suffices for the verification scripts; no optimizer, network access, or third-party Python module is required.

Quick verification. From the extracted archive directory run:

```
python3 -B verify.py
```

This checks the manifest; reconstructs Z215 from the pinned parent using only the two stated edits; verifies the dense bijection and directed state count; replays the complete blank prefix on both tables; reconstructs the exact terminal tape including its zero exterior; checks every ordered saved runtime case and positive affine duration; recomputes the finite scheduling premises in independent programs; checks the source-specific field and halting-test obligations; and forward-checks every inference in the alias-free certificate. It also runs the secondary packet's portable reconstruction and audit suite.

Full symbolic runtime reproduction. To re-execute every symbolic proof case, choose a nonexistent directory:

```
python3 -B replay_runtime.py --out /tmp/z215-runtime
```

This invokes the unchanged inherited symbolic checker with the existing spare append parameter set to zero. The wrapper rederives the four arbitrary-block lemmas and all guarded source cases. It then runs the independent finite audit and compares the deterministic machine, case, and startup artifacts with the delivered copies. The elapsed-time field is naturally machine-dependent. On the research run the symbolic phase took about 133 seconds; this is a reproduction observation, not a complexity claim.

The additional construction has its own commands in `research/command_family/README.md`. Its full symbolic replay is independent of the main machine's replay. The research-only and rejected machines are explicitly labelled in that packet, so a source decoder check cannot be mistaken for blank-tape calibration.

What the checks establish. The full runtime checker constructs an ordinary mathematical proof using explicit induction schemas over arbitrary unary values and finite lists. Its finite execution bound constrains certificate construction, not the values of the simulated registers. The saved-case audit checks the receipts, identities, durations, and complete concrete trace; it does not independently reconstruct every symbolic induction step. The source-focused reviewer uses its own control-graph resolver and direct command-word expansion. The report's unbounded diagonal proof is also separately reviewed in prose.

There is *no new exact-table Lean or Rocq kernel endpoint* in this submission. There is also no independently established internal PA endpoint. Those are distinct deliverables from the ordinary proof formalizable in ZFC. The inherited numerical-calculus theorem is identified explicitly rather than hidden behind a successful script exit code. No finite blank simulation alone is used to establish nonhalting, completeness, or safety of arbitrary raw codes.

Building the paper. With a standard TeX Live installation, run `pdflatexreport.tex` twice from the archive root. The appendix is generated from the dense table by `code/make_machine_table.py`; its delivered text is also manifest-bound. The report is intended to remain useful even without executing the programs: it contains the exact edits, proof interfaces, all-natural scheduling argument, and complete dense table.

9 What changes the search for 214 states

The experiments did not produce a 214-state machine. They do produce two concrete changes to the search space.

First, the physical representation need not preserve the parent's spare value. The old proof already quantified over arbitrary natural spare values; changing the appended value is a parameter change in that invariant, provided the whole engine is rechecked. Z215 realizes this with one write bit and a new usable startup. A future attempt can search for an allocator or engine fold that exploits the fact that new spares are zero, rather than treating every old unit-spare writer as semantically mandatory. Simply removing or redirecting its writer is not justified: arbitrary existing spare values remain in the quantified domain, and decoder visits to the same states must still be handled.

Second, a usable blank seed need not decode into an entirely theorem- valued stack. A finite prefix of safe entries can protect an arbitrary tail if the exact enumeration has a finite certified debt bound. This is a new acceptance criterion for candidate start states and allocations. It matters because a small source or correct primitive engine can still have an unsuitable blank computation. The rejected order-021 table, which actually halts, is a concrete negative control.

The most direct follow-up tasks are:

1. **Exploit the zero-spare engine invariant in allocation.** Search for a real 214-state fold, then prove all primitive cases with the exact new table. State-count synthesis without startup and runtime obligations is insufficient.

2. **Search start states using certified finite debt.** Classify candidate seed stacks by accessible theorem prefixes, not only by whether every decoded entry is already safe. The value two here

is specific to the current command order and field arities.

3. **Relax irrelevant physical preservation.** An alternative engine could recycle or truncate spare blocks while preserving the primary register projection and a sufficient supply of spares. Such a change would require new frame and progress theorems; the current checks preserve the old spare list exactly.

4. **Automate schedule bounds for other field grammars.** The diagonal proof combines command weights, payload arities, and iterated Cantor projections. Synthesizing these bounds could turn many previously rejected startups into proved seeds. The proof must remain about the literal source's decoder.

5. **Complete an alias-free propositional basis.** The 22-node certificate below the dominated row removes reliance on truth and zero aliases for three useful formulas. Deriving the remaining composition/distribution strength and checking full raw-code calibration would be a substantive step toward changing the implication tag. Identity, weakening, and Peirce alone do not complete that project.

6. **Combine the two successful changes only with a fresh proof.** The shared-refill scheduler and zero-valued spare policy are separate proved constructions here. Their naive composition need not retain either blank startup. A new composition needs its own exact prefix and source-seed argument.

7. **Export a kernel-checked exact-table endpoint.** Translate the finite source facts, induction schemas, scheduling argument, and inherited calibration into the repository's formal proof framework. This strengthens assurance without being a state reduction by itself.

The formalism exploration was kept subordinate to its required equiconsistency obligation. Allowing junk codes is useful, but it does not license an unchecked change to implication, generalization, or the theory gate. The main candidate changes only a physical representation and the admissible startup class, so its dependence on the already calibrated all-natural calculus is explicit and narrow.

10 Provenance and attribution

The accepted K215 source, binary engine, dominated numerical calculus, source compiler, and symbolic induction machinery are inherited from the pinned ProveIt repository and its submitted research packets [1, 2, 3, 4]. Their original notices and license texts are retained under `baseline/` and `licenses/`. The shared-CB1 refill idea used by the secondary construction was already present in the latest incoming investigation [5]; this report credits that idea and contributes its new composition, allocation, and full verification.

The inherited notices credit, within their respective scopes, Mira and ProveIt, Elliot Glazer's compact theory work, Jeff's busy-beaver/Rocq reference, Andrew James Wade's compiler ancestry, BusyCoq, and the Coq Library of Undecidability. The current submission does not alter their attribution or relicense their components. The subterm proof-search code also retains the attribution of the repository tools from which it was adapted.

New contributions in this package are the exact zero-spare table and startup; its fresh quantified runtime verification; the current variable-field debt-two theorem and source-specific independent review; the secondary scheduler/command-order composition; the alias-free 22-node certificate and forward checker; and the reproducible artifact and search accounting. Independent reviews refer to separate checking work within this collaboration, not external human endorsement.

A Complete dense transition table

The initial state is 0. The working states are $0, \ldots, 214$ and the halting state is $H = 215$. Entry wDq means write symbol w, move in direction $D \in \{L, R\}$, and enter state q. The two groups of columns are independent continuations of the same table. This appendix is generated directly from `machine/z215.tm`.

State	Read 0	Read 1	State	Read 0	Read 1
0	1R1	1R65	108	0R123	1R9
1	1R93	1R1	109	0R102	1R124
2	0L2	1L3	110	0R152	1R191
3	0L4	1L3	111	0L214	0L96
4	0L6	1L3	112	0R126	1R105
5	1R7	0L5	113	0R76	1R178
6	0L10	1L3	114	0R167	1R185
7	0R7	0R11	115	0R130	0L214
8	0R85	1R195	116	0R150	1R148
9	0R15	1R14	117	0R162	1R206
10	0L16	1L3	118	0L133	0L2
11	0L17	0R11	119	0R40	1R74
12	0L27	1R113	120	0R63	1R134
13	0R20	1R21	121	0R63	1R135
14	0L90	1L32	122	0R72	0L17
15	0L12	0R23	123	0R15	1R134
16	0L24	1L3	124	0R22	1R136
17	1L2	0L2	125	0R149	1R182
18	0R25	0R33	126	0R139	1L21
19	0R207	1R206	127	0R132	1R184
20	0R29	1R30	128	0R210	1R174
21	0R31	1R32	129	0R76	1R180
22	0R25	0R61	130	0R158	1R13
23	0R34	1R23	131	0R209	1R159
24	0L35	1L3	132	0R13	1R143
25	0L36	0R1	133	0R31	1R71
26	0R37	1R26	134	0R40	1L126
27	0R38	1R39	135	0R155	0L17
28	0R40	0L41	136	0R156	0L17
29	0R42	1R43	137	0R89	1R138
30	0R44	1R45	138	0R202	1R142
31	0R46	1R47	139	0R25	0R49
32	0R22	1R48	140	0R142	1R184
33	0R49	1R33	141	0R78	1R95
34	0R33	1R34	142	0R97	1R161
35	0L51	1L3	143	0R179	1R168
36	1R36	1L2	144	0R13	1R161
37	0R52	1R37	145	0R89	1L168
38	0R18	1R115	146	0R89	1R160
39	0R14	1R54	147	0L154	1L165
40	0L12	0R37	148	0R85	1R89
41	0L5	1L55	149	0R161	1R13
42	0R25	0R26	150	0R84	1R145
43	0R14	0L17	151	0R166	1R170

continued on next page

State	Read 0	Read 1	State	Read 0	Read 1
44	0R57	1R58	152	0L68	1L143
45	0R59	1R60	153	0R150	1R208
46	0R25	0R56	154	0R161	1R116
47	0R62	1R63	155	0R98	1R98
48	0R64	0L17	156	0R50	0R33
49	0R26	1R49	157	0R169	0L214
50	0L36	0R65	158	0R18	1R157
51	0L75	1L3	159	0L192	1R97
52	0R17	1R52	160	0R170	1R183
53	0R149	1R145	161	0R20	1R173
54	0R69	1R70	162	0R177	1R167
55	0L41	1L55	163	0R161	1R97
56	0R23	1R56	164	0R127	1R166
57	0L12	0R52	165	0R98	1R164
58	0R62	0L17	166	0L8	1R144
59	0R71	1R72	167	0L110	1R177
60	0R63	1R73	168	0R148	1R176
61	0R56	1R61	169	0R12	1R131
62	0R50	0R26	170	0R163	1R145
63	0L29	1L74	171	0R193	1R167
64	0L119	1L31	172	0R182	1R182
65	0R0	1R65	173	0R185	1R124
66	0R98	1R152	174	0R147	1R167
67	1R189	1L55	175	0R171	1R88
68	0R146	1R147	176	0R76	1R62
69	0R77	1RH	177	0R147	1R89
70	0L202	1R32	178	0R163	1R141
71	0R42	1R64	179	0R142	1R85
72	0L90	1R79	180	0R179	1R76
73	0R57	1L80	181	0R146	1R175
74	0R81	1R82	182	0R144	1L151
75	0R75	1R83	183	0R98	1R53
76	0L147	1L143	184	0R170	1R53
77	0R86	1R87	185	0R18	1L133
78	0R99	1R212	186	0R145	1R181
79	0R90	1L134	187	0R77	1R118
80	0L109	1R58	188	0R107	1R126
81	0R50	0R34	189	0L41	1L111
82	0R92	0L43	190	0R174	1R161
83	0R94	1R203	191	0R152	1R117
84	0R97	1R13	192	0R186	1R187
85	0R89	1R98	193	0R197	1R100
86	0R213	1R22	194	0R148	1R204
87	0R118	1R101	195	0R159	1R196
88	0R152	1R114	196	0R179	1R190
89	0R103	1R104	197	0R66	1R8
90	0R81	1R80	198	0R152	1R129
91	0R102	1R201	199	0R166	1R192
92	0R106	1R107	200	0R154	1R125
93	0R67	0R1	201	0R105	1R97
94	0R28	1R108	202	0R91	1R89

continued on next page

State	Read 0	Read 1	State	Read 0	Read 1
95	0R187	0L214	203	0R109	1R27
96	1L111	1L96	204	0R211	1R110
97	0R20	1R112	205	0R152	1R172
98	0R62	1R62	206	0R205	1R162
99	0R70	1R137	207	0R142	1R140
100	0R68	1R194	208	0R200	1R19
101	0R106	0L214	209	0R202	1R29
102	0R119	1R120	210	0R165	1R198
103	0R73	1R60	211	0R128	1R153
104	0R59	1R121	212	0R199	1R188
105	0R22	1R122	213	0R46	1R101
106	0R22	0L11	214	0L5	0L214
107	0R63	1R43			

References

[1] Vladimir Reshetnikov and contributors. *ProveIt, Computability/BusyBeaver/Research/ZFC*. Snapshot 32498004963846db7655eedce7e94f8c88f6fa8a. https://github.com/VladimirReshetnikov/ProveIt/tree/32498004963846db7655eedce7e94f8c88f6fa8a/Computability/BusyBeaver/Research/ZFC.

[2] ProveIt contributors. *Accepted primary candidate K215: ordinary composition proof, machine, source, and verification*. At the snapshot above, `candidate-k215/`. https://github.com/VladimirReshetnikov/ProveIt/blob/32498004963846db7655eedce7e94f8c88f6fa8a/Computability/BusyBeaver/Research/ZFC/candidate-k215/proof.md.

[3] ProveIt contributors. *Dominated-rho8 numerical calculus and its all-natural consistency calibration*. At the same snapshot, `candidate-k218/alternatives/dominated/proof.md`. https://github.com/VladimirReshetnikov/ProveIt/blob/32498004963846db7655eedce7e94f8c88f6fa8a/Computability/BusyBeaver/Research/ZFC/candidate-k218/alternatives/dominated/proof.md.

[4] ProveIt contributors. *Command-local field reader, permuted commands, and strong source calibration*. At the same snapshot, `candidate-k216/alternatives/permuted/`. The exact inherited LaTeX supplement is retained in `baseline/source_calibration.tex`.

[5] ProveIt contributors. *ZFC214 investigation, 21 October 2026*. `docs/incoming/zfc214-investigation-2026-10-21.zip`, introduced at the pinned snapshot. The relevant scheduler proof sections and exact comparison tables are retained in the secondary research packet.